



MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 8

Optimasi Linear: Linear Programming

Abstrak

Pertemuan ini memperkenalkan Linear Programming (LP) sebagai fondasi optimasi keputusan dengan sumber

Sub - CPMK

Sub-CPMK 3.2 – Mampu mengintegrasikan hasil simulasi untuk mendukung keputusan desain

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

🔗 **Modul Interaktif:** <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-8.html>. Pada layar masuk, pilih tombol “👁️ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Setelah Ujian Tengah Semester, mata kuliah memasuki bagian kedua: optimasi. Pertemuan kesembilan ini membahas Linear Programming (LP), teknik optimasi klasik untuk memaksimalkan profit atau meminimalkan biaya di bawah kendala sumber daya yang linier. LP menjadi fondasi sebelum masuk ke optimasi non-linear (Pertemuan 10) dan metaheuristik (Pertemuan 11).

Posisi Pertemuan 9 dalam Alur Mata Kuliah Optimalisasi & Otomasi



Gambar 1. Posisi Pertemuan 9 (Linear Programming) dalam alur mata kuliah, mengawali blok materi optimasi setelah UTS.

Gambar 1 menunjukkan posisi Pertemuan 9 dalam alur mata kuliah, menjembatani materi analisis sinyal (Pertemuan 1-7) dengan materi optimasi dan monitoring cerdas (Pertemuan 10-15).

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 3.2:** Mampu mengintegrasikan hasil simulasi untuk mendukung keputusan desain, yaitu menggunakan hasil solve Linear Programming (nilai optimal, shadow price, sensitivity analysis) sebagai dasar pengambilan keputusan alokasi sumber daya dalam desain sistem produksi (CPMK 3).
- Setelah pertemuan ini mahasiswa dapat: memformulasikan masalah rekayasa sebagai LP standar; menyelesaikan LP dua variabel secara grafis; memahami prinsip kerja algoritma Simpleks; serta menginterpretasikan shadow price dari sensitivity analysis.

MENGAPA LINEAR PROGRAMMING

$$Z = c_1x_1 + c_2x_2 + \dots + c_nx_n \quad P = \{x \mid Ax \leq b, x \geq 0\} \quad x^* \in \text{vertices}(P)$$

Asumsi Fundamental LP: LP mengasumsikan baik fungsi objektif maupun seluruh kendala bersifat linier terhadap variabel keputusan; variabel bersifat kontinu dan umumnya non-negatif. Daerah feasibel yang terbentuk dari irisan kendala linier selalu berupa poligon konveks (polihedron di dimensi lebih tinggi).

Teorema Dasar LP: Teorema Dasar Linear Programming (Dantzig, 1947) menyatakan bahwa solusi optimal LP, jika daerah feasibelnya terbatas, selalu berada di setidaknya satu vertex (titik sudut) daerah feasibel, bukan di titik interior. Inilah dasar mengapa algoritma Simpleks cukup menelusuri vertex demi vertex, bukan seluruh titik di dalam daerah feasibel.

Implementasi Python: Implementasi praktis LP di Python memakai ``scipy.optimize.linprog``, yang secara default meminimumkan fungsi objektif; untuk maksimisasi, koefisien c harus dinegasikan terlebih dahulu.

FORMULASI STANDAR LP

Bentuk Kanonik: $\max c^T x$ s.t. $Ax \leq b, x \geq 0$ *Bentuk Standar (Slack): $Ax + Is = b, x, s \geq 0$*
Bounds: $lb \leq x \leq ub$

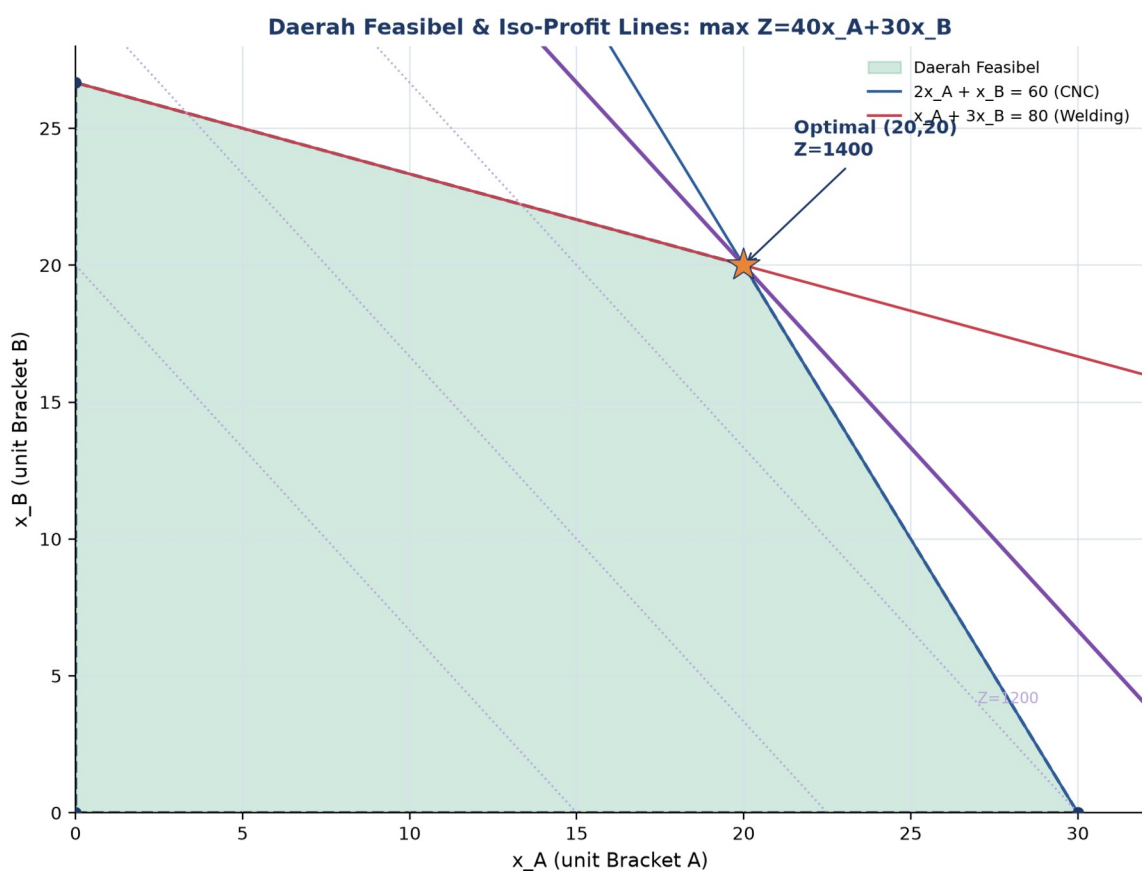
- **Bentuk Standar (Slack Variables):** kendala inequality ($\leq$) diubah menjadi equality dengan menambah slack variable non-negatif; ``Ax + Is = b``.
- **Bounds: Variabel Berbatas:** batas atas/bawah tiap variabel diset lewat parameter ``bounds`` di `linprog`, terpisah dari kendala ``A_ub`/`b_ub``.
- **Tiga Status Solusi LP:** status 0 = optimal ditemukan; status 2 = infeasible (daerah feasibel kosong); status 3 = unbounded (Z tak terbatas).

Contoh Kasus: Pabrik Komponen Mesin. Pabrik komponen mesin memproduksi dua jenis bracket: Bracket A memberi profit Rp40 ribu/unit, membutuhkan 2 jam CNC dan 1 jam welding; Bracket B memberi profit Rp30 ribu/unit, membutuhkan 1 jam CNC dan 3 jam welding. Kapasitas CNC 60 jam/minggu, kapasitas welding 80 jam/minggu. Formulasi: maksimalkan $Z = 40x_A + 30x_B$ dengan kendala $2x_A + x_B \leq 60$ dan $x_A + 3x_B \leq 80, x \geq 0$. Solusi optimal berada di vertex (20,20) dengan $Z = \text{Rp}1.400$ ribu.

Cheat Sheet Konversi: Konversi bentuk umum ke bentuk standar linprog: $\min \leftrightarrow \max$ dengan negasi c ; kendala $\geq$ dikonversi ke $\leq$ dengan negasi kedua sisi; kendala $=$ tetap dipisah sebagai ``A_eq`/`b_eq``; variabel bebas tanda dipecah menjadi dua variabel non-negatif; selalu periksa konsistensi dimensi A, b , dan bounds sebelum menjalankan solver.

DAERAH FEASIBEL & METODE GRAFIS

- **Plot Constraint Lines:** gambar tiap kendala sebagai garis batas di bidang 2D.
- **Identifikasi Daerah Feasibel:** irisan seluruh half-plane kendala membentuk daerah feasibel berupa poligon konveks.
- **Iso-Profit Line & Optimasi:** garis $Z=\text{konstan}$ digeser sejajar ke arah peningkatan objektif hingga menyentuh vertex terakhir daerah feasibel; itulah solusi optimal.
- **Enumerasi Vertex (Brute Force):** cara brute-force: evaluasi Z di setiap vertex daerah feasibel, lalu pilih yang terbesar (maksimisasi) atau terkecil (minimisasi).



Gambar 2. Daerah feasibel (hijau) dan garis iso-profit untuk masalah pabrik bracket; solusi optimal di vertex $(20,20)$ dengan $Z=1400$.

Gambar 2 mengilustrasikan metode grafis: garis iso-profit digeser ke kanan-atas hingga menyentuh vertex terakhir sebelum keluar dari daerah feasibel, yaitu titik $(20,20)$.

Keterbatasan Metode Grafis: Metode grafis hanya praktis untuk masalah dua variabel keputusan karena keterbatasan visualisasi bidang; begitu jumlah variabel mencapai tiga, daerah feasibel menjadi polihedron tiga dimensi yang masih dapat digambar namun sudah sulit dibaca secara visual, dan untuk empat variabel atau lebih, representasi geometris langsung sudah tidak lagi memungkinkan. Untuk kasus itulah metode Simpleks (bergerak

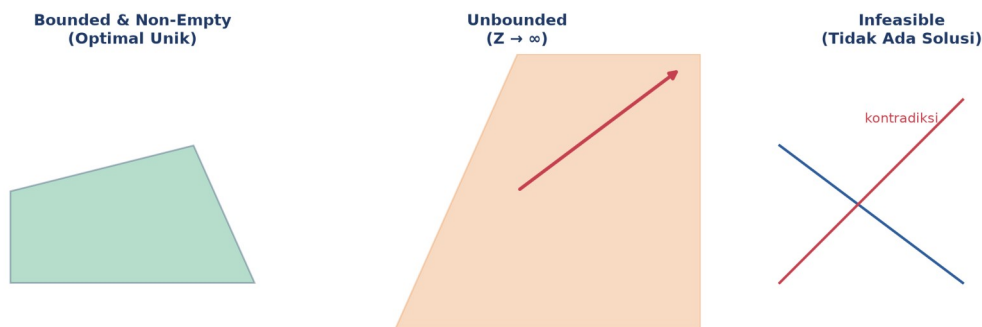
antar vertex secara aljabar tanpa perlu visualisasi) dan solver numerik seperti HiGHS menjadi pendekatan standar industri, sementara metode grafis tetap bernilai pedagogis penting karena memberi intuisi visual tentang MENGAPA solusi optimal LP selalu berada di salah satu vertex daerah feasibel, bukan di titik interior sembarang.

Tabel 1. Tiga status utama daerah feasibel LP beserta karakteristik, solusi, dan aksi yang perlu diambil engineer.

Status Daerah Feasibel	Karakteristik	Solusi LP	Aksi Engineer
Bounded & non-empty	Poligon tertutup terbatas	Optimal unik (atau edge multiple)	Lanjut solve dengan Simpleks/HiGHS
Unbounded	Membentang ke tak hingga pada arah c	Z menuju tak hingga (unbounded)	Tambah kendala, formulasi kurang lengkap
Empty (infeasible)	Tidak ada x yang memenuhi semua kendala	Tidak ada solusi	Cek kontradiksi kendala
Single point	Titik tunggal (semua kendala equality)	Solusi tunggal trivial	Tidak perlu optimasi, substitusi langsung
Degenerate vertex	3 atau lebih kendala bertemu di 1 titik	Optimal tapi berisiko cycling	Pakai aturan Bland di Simpleks

Tabel 1 merangkum status yang mungkin terjadi pada daerah feasibel LP.

Tiga Status Daerah Feasibel LP



Gambar 3. Ilustrasi visual tiga status utama daerah feasibel: bounded & non-empty, unbounded, dan infeasible.

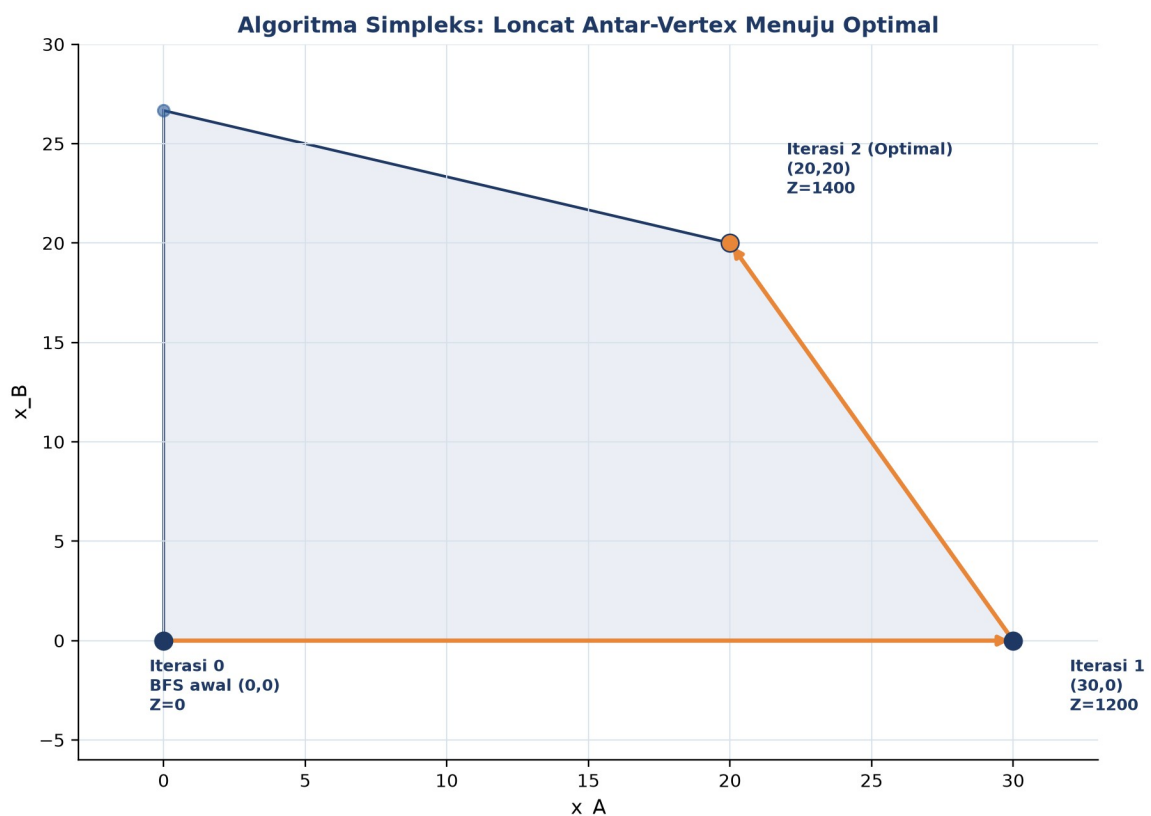
Gambar 3 memvisualisasikan tiga status tersebut secara geometris; kasus unbounded dan infeasible biasanya menandakan formulasi kendala yang kurang lengkap atau kontradiktif.

METODE SIMPLEKS

Algoritma Simpleks dikembangkan oleh George Dantzig pada tahun 1947 untuk kebutuhan logistik militer Angkatan Udara AS. Meski Klee-Minty (1972) menunjukkan kompleksitas worst-case eksponensial, dalam praktik Simpleks sangat cepat dan tetap dipakai luas hingga kini.

Initial BFS Pricing: $\bar{z}_j = c_j - c_{BV} \cdot B^{-1} \cdot a_j$ Ratio Test: $\theta^ = \min\{b_i/a_{ij} : a_{ij} > 0\}$
Pivot (Gauss-Jordan)*

- **Tableau Awal & BFS:** mulai dari basic feasible solution awal, biasanya origin dengan semua slack variable sebagai variabel basis.
- **Pricing: Pilih Variabel Masuk:** pilih variabel non-basis dengan reduced cost paling menguntungkan sebagai variabel masuk.
- **Ratio Test: Pilih Variabel Keluar:** tentukan variabel keluar lewat rasio minimum b_i/a_{ij} agar tetap feasibel.
- **Pivot: Update Tableau:** update tableau via eliminasi Gauss-Jordan; nilai Z tidak pernah menurun di tiap iterasi.
- **Big-M & Two-Phase Method:** untuk kendala equality/ $\geq$ yang tidak punya BFS trivial, dipakai variabel artifisial dengan penalti besar M, atau pendekatan dua fase.
- **Revised Simplex & Modern Solver:** solver modern (HiGHS, dipakai ``scipy.optimize.linprog(method='highs')``) memakai revised Simplex atau interior-point method yang jauh lebih efisien untuk masalah skala besar.



Gambar 4. Algoritma Simpleks bergerak dari vertex ke vertex tetangga: $(0,0)$ dengan $Z=0$, ke $(30,0)$ dengan $Z=1200$, ke $(20,20)$ dengan $Z=1400$ (optimal).

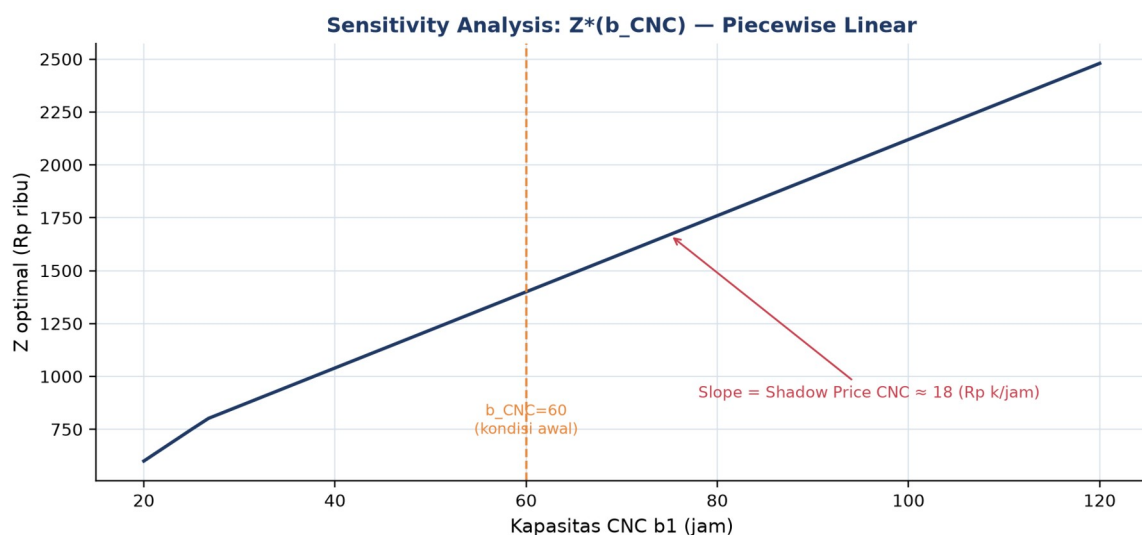
Gambar 4 mengilustrasikan bagaimana Simpleks meloncat dari vertex awal (0,0) menuju vertex optimal (20,20) melalui satu vertex perantara, dengan nilai Z yang selalu meningkat setiap iterasi.

Tabel 2. Perbandingan tiga pendekatan solver LP: Simpleks, Interior Point, dan Network Simplex.

Aspek	Simpleks	Interior Point	Network Simplex
Worst-case	Eksponensial (Klee-Minty)	$O(n^3 L)$ polynomial	$O(mn)$ untuk network LP
Praktik	Sekitar 2m-3m iterasi, sangat cepat	Sekitar 30-50 iterasi, scalable	10-100 kali lebih cepat untuk transport
Solusi	Vertex (sparse)	Interior (dense)	Vertex jaringan
Cocok untuk	LP kecil-sedang, warm start	LP besar, dense matrix	Transportation, shortest path
Library Python	linprog(method='simplex')	linprog(method='highs-ipm')	networkx / linear_sum_assignment

Tabel 2 membandingkan tiga pendekatan solver LP dari sisi kompleksitas, karakteristik solusi, dan library Python yang relevan.

Dual Simplex & Sensitivity Analysis: Untuk masalah pabrik bracket, dual value (shadow price) dari sistem persamaan $2y_1 + y_2 = 40$ dan $y_1 + 3y_2 = 30$ menghasilkan $y_{\text{CNC}} = 18$ dan $y_{\text{welding}} = 4$; artinya tiap tambahan 1 jam kapasitas CNC menaikkan profit optimal sekitar Rp18 ribu, sedangkan tambahan 1 jam welding hanya menaikkan sekitar Rp4 ribu.



Gambar 5. Sensitivity analysis $Z^*(b_{\text{CNC}})$: fungsi piecewise linear terhadap kapasitas CNC, dengan slope sama dengan shadow price.

Gambar 5 menunjukkan bagaimana Z optimal berubah secara piecewise linear terhadap kapasitas CNC; kemiringan garis pada rentang di sekitar $b_{\text{CNC}} = 60$ sama dengan shadow price CNC.

Peringatan, Degenerasi & Cycling: Degenerasi terjadi ketika lebih dari n kendala aktif bertemu di satu vertex, berisiko menyebabkan cycling (Simpleks berputar tanpa progres). Aturan Bland (memilih variabel dengan indeks terkecil saat terjadi seri) menjamin terminasi meski memperlambat konvergensi.

ANALOGI KEHIDUPAN SEHARI-HARI

- **Diet Plan & Stigler's Diet Problem:** Stigler (1939) menyelesaikan manual diet termurah seharga \$39.93/tahun; setelah Simpleks ditemukan Dantzig (1947) menghitung ulang menjadi \$39.69/tahun.
- **Logistik & Transportation Problem:** rute pengiriman gudang Amazon adalah LP klasik dengan variabel keputusan jumlah barang yang dikirim per rute.
- **Refinery Blending & Petroleum LP:** Shell pada 1950-an menjadi pionir LP untuk blending fraksi minyak mentah menjadi produk bahan bakar.
- **Penjadwalan Mesin di Pabrik:** penjadwalan 10 mesin CNC untuk 50 jenis produk adalah LP/MILP skala industri, kunci Industry 4.0.
- **Portfolio & Markowitz's Model:** model Markowitz (1952) mengalokasikan portofolio investasi via LP/QP, mendasari robo-advisor modern.
- **Power Grid & Economic Dispatch:** PLN P2B menjalankan LP setiap 5 menit untuk economic dispatch pembangkit listrik.

Pola Umum: Pola yang sama muncul di semua analogi: sumber daya terbatas, banyak pilihan alokasi, dan kebutuhan mencari titik optimal secara sistematis, bukan coba-coba.

APLIKASI DI TEKNIK MESIN & INDUSTRI 4.0

- **Production Planning & Mix Optimization:** Toyota, Bosch, dan Astra memakai LP untuk menentukan bauran produksi optimal dari 8 jenis bracket dengan software SAP IBP.
- **Material Blending: Steel & Alloy:** ArcelorMittal, Krakatau Steel, dan Posco memakai LP untuk blending scrap/pig iron/ferro-alloy pada tanur EAF agar komposisi kimia sesuai target dengan biaya minimum.
- **Logistik & Vehicle Routing:** perusahaan logistik seperti Logivan dan Amazon FBA memakai LP untuk mengalokasikan 500 ton/minggu dari 3 gudang ke 12 pabrik dengan biaya transportasi minimum.
- **Energy Optimization Smart Factory:** smart factory dengan solar PV 200kW, genset, dan grid PLN (kontrak maksimum 150kW) memakai LP untuk menjadwalkan sumber

energi termurah tiap jam, mengikuti pendekatan Siemens MindSphere dan GE Predix.

Peringatan, Jebakan Praktis Formulasi LP: Kesalahan praktis yang sering terjadi: lupa mengecek satuan konsisten antar kendala; kendala yang saling kontradiktif menghasilkan infeasible tanpa disadari; lupa menambah bounds non-negatif; menganggap hasil LP selalu integer padahal variabel kontinu; tidak melakukan sensitivity analysis meski keputusan bisnis sensitif terhadap kapasitas; serta salah interpretasi shadow price pada kendala yang tidak binding (slack>0).

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada studi kasus pabrik bracket dan perluasannya ke skala industri. Lingkungan: conda env optoauto dengan paket numpy, scipy, matplotlib, pandas; notebook p9_linear_programming.ipynb.

p9_linear_programming.ipynb — Cell 1

```
from scipy.optimize import linprog

c = [-40, -30]                # maksimisasi: negasikan
A_ub = [[2, 1], [1, 3]]      # CNC, Welding
b_ub = [60, 80]
bounds = [(0, None), (0, None)]

res = linprog(c, A_ub=A_ub, b_ub=b_ub,
              bounds=bounds, method='highs')

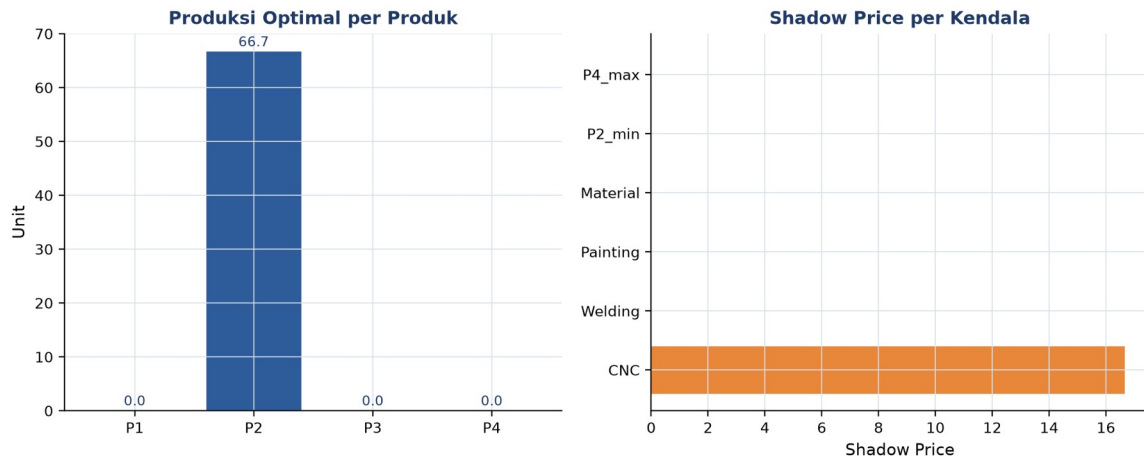
print(f'x_A={res.x[0]:.1f}, x_B={res.x[1]:.1f}')
print(f'Z_max = {-res.fun:.0f}')
# output: x_A=20.0, x_B=20.0, Z_max=1400
```

Gambar 6. Potongan kode formulasi dan penyelesaian LP pabrik bracket menggunakan `scipy.optimize.linprog`.

Gambar 6 menunjukkan potongan kode inti Cell 1.

- **Cell 1:** formulasi LP pabrik bracket: $\max 40x_A + 30x_B$ s.t. kendala CNC dan welding; solve dengan `linprog(method='highs')`; cetak `x_A`, `x_B`, `Z_max`, dan utilisasi tiap sumber daya.
- **Cell 2:** visualisasi daerah feasibel via meshgrid, garis kendala, garis iso-profit, enumerasi 4 vertex, dan penanda vertex optimal.
- **Cell 3:** LP skala industri 4 produk x 6 kendala (CNC, welding, painting, material, batas minimum P2, batas maksimum P4) plus sensitivity analysis: shadow price dari `-result.ineqlin.marginals` dan slack dari `result.ineqlin.residual`.

- **Cell 4:** fungsi ``solve_production_lp()`` reusable mengembalikan dict berisi status, Z, produksi, shadow price, utilisasi, dan slack sebagai pandas Series; divisualisasikan sebagai bar chart produksi dan shadow price.



Gambar 7. Hasil LP 4-produk skala industri: produksi optimal per produk (kiri) dan shadow price per kendala (kanan) dari Cell 3-4.

Gambar 7 menunjukkan hasil LP 4-produk dari Cell 3-4: hanya produk P2 yang diproduksi pada solusi optimal, dan hanya kendala CNC yang binding (shadow price positif), sedangkan kendala lain memiliki slack sehingga shadow price-nya nol.

TUGAS PERTEMUAN 9

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

Distribusi Bobot Tugas Pertemuan 9



Gambar 8. Distribusi 50 poin Tugas Pertemuan 9 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Pernyataan yang BENAR mengenai asumsi fundamental Linear Programming adalah...

- A. Fungsi objektif boleh kuadratik tapi kendala harus linier
- B. Baik fungsi objektif maupun kendala harus linier dalam variabel keputusan; variabel kontinu dan biasanya non-negatif
- C. Variabel keputusan harus integer, tidak boleh pecahan
- D. LP hanya bisa diselesaikan untuk masalah 2 variabel

2. Teorema Dasar Linear Programming menyatakan bahwa solusi optimal LP (jika terbatas) selalu berada di...

- A. Titik tengah daerah feasibel
- B. Setidaknya satu vertex (titik sudut) daerah feasibel
- C. Origin $x = 0$ selalu
- D. Bidang dengan luasan terbesar

3. Tujuan menambahkan slack variable $s_i \geq 0$ ke kendala $a_i^T x \leq b_i$ adalah...

- A. Memperkenalkan ketidakpastian probabilistik ke LP
- B. Mengkonversi kendala inequality menjadi equality $a_i^T x + s_i = b_i$ agar bisa diolah algoritma Simpleks
- C. Membuat masalah jadi non-linier
- D. Memperbesar daerah feasibel dengan tambahan dimensi

4. Pada `scipy.optimize.linprog`, untuk memecahkan masalah maksimisasi $\max Z = c^T x$, yang harus dilakukan adalah...

- A. Memberi parameter `direction='max'` ke `linprog`
- B. Negasikan koefisien c (kirim $-c$) karena `linprog` selalu meminimumkan; nilai Z dikembalikan dengan `-result.fun`
- C. Tidak bisa, `scipy` tidak mendukung maksimisasi sama sekali
- D. Membalikkan tanda b_{ub} dan A_{ub} saja

5. Setelah solver LP berjalan, status "unbounded" berarti...

- A. Solusi optimal sudah ditemukan dengan akurasi tinggi
- B. Daerah feasibel terbentang ke arah peningkatan Z tanpa batas, biasanya kurang kendala fisik; harus ditinjau ulang formulasinya
- C. Daerah feasibel kosong, kendala kontradiktif
- D. Solver belum konvergen, perlu iterasi lebih banyak

6. Daerah feasibel sebuah masalah LP selalu berbentuk...

- A. Lingkaran (lingkup bola di dimensi tinggi)
- B. Polihedron (poligon konveks di 2D, polihedron di nD), irisan setengah-bidang
- C. Selalu cembung tetapi tidak pernah terbatas (bounded)

- D. Bentuk fraktal yang kompleks

7. Algoritma Simpleks menyelesaikan LP dengan cara...

- A. Memeriksa semua titik di dalam daerah feasibel satu per satu
- B. Bergerak dari satu vertex ke vertex tetangga yang memberi nilai objektif lebih baik, hingga tidak ada peningkatan lagi
- C. Mengambil rata-rata dari semua kendala
- D. Menebak solusi awal lalu menggunakan gradient descent kontinu

8. Shadow price (dual variable) sebuah kendala mengindikasikan...

- A. Total biaya yang dikeluarkan untuk kendala tersebut
- B. Berapa peningkatan nilai Z optimal kalau RHS kendala bi ditambah 1 unit, sebagai panduan investasi sumber daya
- C. Probabilitas kendala tidak terpenuhi
- D. Bobot kendala dalam fungsi objektif

9. Untuk LP $\min Z = 3x_1 + 5x_2$ s.t. $x_1 + x_2 \geq 4$, $x \geq 0$, agar bisa diolah linprog, kendala $\geq$ harus dikonversi menjadi...

- A. $x_1 + x_2 = 4$ (langsung dijadikan equality)
- B. $-x_1 - x_2 \leq -4$ (negasikan kedua sisi membalik tanda inequality)
- C. Menambah variabel kuadratik di sisi kiri
- D. Tidak perlu konversi, kendala $\geq$ langsung diterima

10. Untuk masalah pabrik dengan kebutuhan jumlah unit produksi harus integer (misal tidak bisa produksi 3,7 mesin), tool yang paling tepat adalah...

- A. Tetap pakai LP biasa, lalu pembulatan hasil ke integer terdekat
- B. Mixed-Integer Linear Programming (MILP) dengan variabel bertipe integer (integrality=1 di scipy 1.9+); pembulatan LP biasa bisa infeasible atau sub-optimal
- C. Quadratic Programming saja
- D. LP dengan kendala tambahan x anggota bilangan real

Bagian B: Komputasi Easy-Medium (10 soal $\times$ 2 poin)

Catatan: gunakan `scipy.optimize.linprog` dengan `method='highs'`. Ingat scipy selalu meminimumkan, untuk maksimisasi negasikan koefisien c. Hasil profit dikembalikan dengan `-result.fun`. Pastikan cek `result.success` sebelum membaca `result.x`.

```
from scipy.optimize import linprog
# Bentuk umum: minimize c.x s.t. A_ub.x <= b_ub, bounds
# Maksimisasi: kirim -c, ambil Z = -result.fun
```

C1. Selesaikan LP 1D sederhana: $\max Z = 5x$ s.t. $2x \leq 10$, $x \geq 0$. Print nilai Z optimal.

-  **HINT:** linprog meminimalkan, jadi untuk maksimisasi negasikan vektor c; nilai Z optimal didapat dari `-res.fun`.

C2. Pabrik bracket 2-produk: $\max Z = 40x_A + 30x_B$ s.t. $2x_A + x_B \leq 60$ (CNC), $x_A + 3x_B \leq 80$ (welding), $x \geq 0$. Selesaikan dengan linprog dan print Z optimal (Rp ribu).

-  **HINT:** Susun c (negasi karena maksimisasi), A_ub dari koefisien kendala, dan b_ub dari batas kapasitas; ambil Z dari -res.fun.

C3. Untuk LP yang sama dengan soal sebelumnya, print x_A optimal (jumlah unit Bracket A).

-  **HINT:** Vertex optimal terjadi di perpotongan kedua kendala binding (CNC dan welding); nilai x_A diambil dari elemen pertama res.x.

C4. LP minimisasi diet: $\min Z = 2x_1 + 3x_2$ (biaya per unit) s.t. $x_1 + 2x_2 \geq 10$ (protein), $3x_1 + x_2 \geq 15$ (kalori), $x \geq 0$. Print Z minimum.

-  **HINT:** Konversi kendala $\geq$ menjadi $\leq$ dengan menegasikan kedua sisi; karena ini minimisasi, c dipakai langsung tanpa negasi dan $Z = \text{res.fun}$.

C5. LP 3 variabel: $\max Z = 10x_1 + 6x_2 + 4x_3$ s.t. $x_1 + x_2 + x_3 \leq 100$, $10x_1 + 4x_2 + 5x_3 \leq 600$, $2x_1 + 2x_2 + 6x_3 \leq 300$, $x \geq 0$. Print Z optimal.

-  **HINT:** Susun c (negasi untuk maksimisasi) dan tiga baris A_ub/b_ub dari kendala; serahkan ke linprog dan ambil Z dari -res.fun.

C6. Slack di vertex optimal pabrik bracket 2-produk: hitung slack kendala CNC $s_{\text{CNC}} = 60 - (2x_A + x_B)$ di solusi optimal (selesaikan LP tersebut lebih dulu). Print nilainya.

-  **HINT:** Slack = kapasitas dikurangi pemakaian di solusi optimal; slack = 0 berarti kendala binding. Bisa juga dibaca dari res.ineqlin.residual.

C7. Hitung shadow price untuk kendala CNC pada masalah pabrik bracket 2-produk. Pakai res.ineqlin.marginals lalu negasikan (karena scipy negasi internal). Print nilainya (Rp ribu/jam).

-  **HINT:** Shadow price = perubahan Z per unit kenaikan kapasitas kendala; baca dari res.ineqlin.marginals lalu negasikan karena konvensi internal scipy.

C8. LP dengan kendala equality: $\min Z = x_1 + 2x_2$ s.t. $x_1 + x_2 = 10$, $x_1 \geq 0$, $x_2 \geq 0$. Pakai parameter A_eq, b_eq di linprog. Print Z minimum.

-  **HINT:** Gunakan parameter A_eq dan b_eq untuk kendala equality; karena minimisasi, solver memilih variabel berkoefisien kecil. Ambil Z dari res.fun.

C9. Vertex enumeration manual: untuk LP $\max Z = 3x_1 + 2x_2$ s.t. $x_1 + x_2 \leq 4$, $x_1 \leq 3$, $x \geq 0$. Vertex feasibel: (0,0), (3,0), (3,1), (0,4). Hitung Z maksimal dengan iterasi loop di Python.

-  **HINT:** Loop tiap vertex feasibel yang diberikan, evaluasi fungsi tujuan $Z = 3x_1 + 2x_2$, lalu ambil nilai maksimum dengan max.

C10. Pabrik dengan bounds variabel: $\max Z = 5x_1 + 4x_2$ s.t. $x_1 + x_2 \leq 50$, $0 \leq x_1 \leq 20$, $0 \leq x_2 \leq 35$. Pakai parameter bounds di linprog. Print Z optimal.

-  **HINT:** Masukkan batas atas/bawah variabel lewat parameter bounds; negasikan c untuk maksimisasi dan ambil Z dari -res.fun.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan 20-50+ baris kode dengan algoritma lengkap dari awal. Hint minimal, mahasiswa diharapkan meneliti dan menulis sendiri. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil tidak sesuai/error): +1 poin partial. Kode kosong + submit = 0 poin dan tidak terkunci (bisa retry).

C11 (Hard). LP 4-Produk dengan 6 Kendala: $\max Z = 30x_1 + 50x_2 + 40x_3 + 60x_4$ s.t. CNC: $2x_1 + 3x_2 + 2,5x_3 + 4x_4 \leq 200$; Welding: $x_1 + 2x_2 + 1,5x_3 + 3x_4 \leq 150$; Painting: $0,5x_1 + x_2 + x_3 + 2x_4 \leq 120$; Material: $5x_1 + 8x_2 + 6x_3 + 10x_4 \leq 800$; $x_2 \geq 5$; $x_4 \leq 30$. Print Z optimal.

-  **HINT:** Negasikan c untuk maksimisasi; kendala $x_2 \geq 5$ ditulis sebagai $-x_2 \leq -5$ dan $x_4 \leq 30$ sebagai baris biasa. Ambil Z dari -res.fun.

C12 (Hard). Transportation Problem: 2 gudang (kapasitas 30, 40) kirim ke 3 kota (demand 20, 25, 25). Cost matrix (Rp ribu per unit): $C = \begin{bmatrix} 8, 6, 10 \\ 9, 12, 13 \end{bmatrix}$. Variabel x_{ij} = unit dari gudang i ke kota j (6 variabel). Print total cost minimum.

-  **HINT:** Flatten cost matrix menjadi vektor c, susun baris kendala supply per gudang dan demand per kota; karena total supply = total demand, pakai kendala equality. Ambil biaya dari res.fun.

C13 (Hard). Blending Problem (Steel Alloy): campur 3 raw material x_1, x_2, x_3 (harga Rp/kg: 10, 8, 12) untuk hasilkan 100 kg alloy. Kandungan karbon (%): [0,5; 0,3; 0,7]; target karbon $\geq 0,5\%$. Minimalkan total biaya. Print biaya minimum (Rp/100 kg).

-  **HINT:** Kendala total berat = 100 kg pakai equality; kendala karbon $\geq$ target dikonversi ke $\leq$ dengan negasi. Karena minimisasi, biaya diambil dari res.fun.

C14 (Hard). Vertex enumerasi LP 2D otomatis: untuk LP $\max Z = 4x_1 + 3x_2$ s.t. $2x_1 + x_2 \leq 8$, $x_1 + 2x_2 \leq 8$, $x \geq 0$: tulis algoritma yang (1) cari semua titik perpotongan tiap pasangan kendala (termasuk sumbu), (2) filter feasibel, (3) evaluasi Z di tiap vertex feasibel, (4) pilih max. Print Z optimal.

-  **HINT:** Cari titik perpotongan tiap pasang garis kendala (termasuk sumbu) dengan np.linalg.solve, filter yang feasibel, lalu evaluasi Z di tiap vertex dan ambil maksimum.

C15 (Hard). Sensitivity Analysis: untuk pabrik bracket 2-produk ($\max 40x_A + 30x_B$, kendala $\text{CNC} \leq 60$, $\text{Welding} \leq 80$, $x \geq 0$). Hitung perubahan Z optimal kalau kapasitas CNC ditambah dari 60 menjadi 65 (lebih 5 jam). Bandingkan dengan prediksi shadow price ($\Delta \text{kapasitas} \times \text{shadow_price_CNC}$). Print Z_{baru} minus Z_{lama} .

- 💡 **HINT:** Selesaikan LP dua kali (kapasitas CNC lama vs baru), lalu hitung selisih Z; bandingkan dengan prediksi shadow price dikali Δb sebagai validasi sensitivity analysis.

DAFTAR PUSTAKA

- Bayá, G., Canale, E., Nesmachnow, S., De Sousa, A., & Sartor, P. (2022). Production optimization in a grain facility through mixed-integer linear programming. *Applied Sciences*, 12(16), 8212. <https://doi.org/10.3390/app12168212>
- Oancea, G. (2022). Machining parameters optimization based on objective function linearization. *Mathematics*, 10(5), 803. <https://doi.org/10.3390/math10050803>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>